

Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение
высшего образования
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

ФАКУЛЬТЕТ ПРИКЛАДНОЙ ИНФОРМАТИКИ

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ
по дисциплине «Мобильные и сетевые технологии»
НА ТЕМУ

Графы
15 Вариант

студент Северюхин Андрей Аркадьевич K3341
(Фамилия, Имя, Отчество) (группа)

Направление подготовки (специальность) 09.03.03
(код, наименование)

Мобильные и сетевые технологии

Квалификация Бакалавр
(бакалавр, магистр)*

Преподаватель Ромакина О.М.
(Фамилия, И. О.) (Подпись)

Дата выполнения: 21.09.2026

Санкт-Петербург

2026 г.

Формат представления графов в некоторых заданиях

В задачах 1,2... графы даются на ввод следующим образом. В первой строке заданы целые неотрицательные числа n и m – количество вершин и количество ребер соответственно. Вершины всегда нумеруются от 1 до n . Каждая из следующих m строк определяет ребро в формате $u\ v$ через пробел, где $1 \leq u, v \leq n$ – инцидентные вершины ребра. Если в задаче дан неориентированный граф, то такая запись задает неориентированное ребро между вершинами u и v . В случае ориентированного графа – определяет направленное ребро от u к v . Если в задаче дан взвешенный граф, то каждое ребро задается как $u\ v\ w$, где u и v – вершины, а w – вес ребра. Гарантируется, что данный граф является простым. То есть он не содержит петель (ребер, идущих от вершины к самой себе) и кратных ребер.

5 задача.

Город с односторонним движением (1.5 балла)

Департамент полиции города сделал все улицы односторонними. Вы хотели бы проверить, можно ли законно проехать с любого перекрестка на какой-либо другой перекресток. Для этого строится ориентированный граф: вершины – это перекрестки, существует ребро (u, v) всякий раз, когда в городе есть улица (с односторонним движением) из u в v . Тогда достаточно проверить, все ли вершины графа лежат в одном компоненте сильной связности.

Нужно вычислить количество компонентов сильной связности заданного ориентированного графа с n вершинами и m ребрами.

Формат ввода / входного файла (input.txt). Ориентированный граф с n вершинами и m ребрами по формату 1.

Ограничения на входные данные.

$1 \leq n \leq 10^4$, $0 \leq m \leq 10^4$.

Формат вывода / выходного файла (output.txt). Выведите число – количество компонентов сильной связности.

Ограничение по времени. 5 сек.

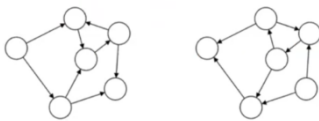
Ограничение по памяти. 512 мб.

Описание решения

Задача разобрана в лекции 5, раздел "Сильно связанные компоненты". Воспользуемся решением.

Сильно связанные компоненты: обратный граф

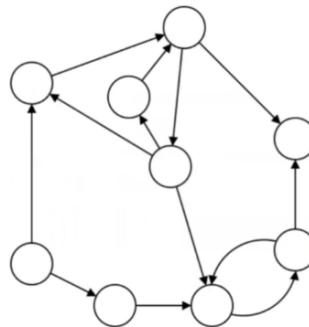
- Пусть граф $G^T = (V, E^T)$ – граф, полученный из G путем изменения направления всех ребер в G на обратное (*транспонирование* графа).
- Для представления с использованием *списка смежности* такой граф можно получить за линейное время $O(|V| + |E|)$.



- Графы G и G^T имеют одни и те же сильно связанные компоненты.
- Компоненты-источники в G^T – это компоненты-стоки в графе G .
- Нужно найти компоненты-стоки в графе G – запуская поиск в глубину (DFS) на G^T .

Сильно связанные компоненты: алгоритм

```
SCC(G):
1  Запустим DFS( $G^T$ )
2  for v in V при убывании post[v]:
3      if not visited[v]:
4          Explore(v)
5      Отметим посещенные вершины
      как новый компонент SCC
```



Если запустить Explore из вершины компоненты-стока, обход соберет ровно эту компоненту и уйти из стока некуда. Значит нужны стоки, но напрямую их найти не получится. По теореме вершина с наибольшим post лежит в компоненте-истоке, а истоки обратного графа – это стоки исходного, компоненты у них общие.

Асимптотика $O(n + m)$

Решение

```
n, m = map(int, input().split())
g = [[] for _ in range(n + 1)]
gt = [[] for _ in range(n + 1)]

for _ in range(m):
    u, v = map(int, input().split())
    g[u].append(v)
    gt[v].append(u)

visited = [False] * (n + 1)
order = []

for s in range(1, n + 1):
    if visited[s]:
        continue
    stack = [(s, False)]
    while stack:
        v, done = stack.pop()
        if done:
            order.append(v)
            continue
        if visited[v]:
            continue
        visited[v] = True
        stack.append((v, True))
        for u in gt[v]:
            if not visited[u]:
                stack.append((u, False))

visited = [False] * (n + 1)
count = 0

for v in reversed(order):
    if visited[v]:
        continue
    count += 1
    visited[v] = True
    stack = [v]
    while stack:
        x = stack.pop()
        for u in g[x]:
            if not visited[u]:
                visited[u] = True
                stack.append(u)

print(count)
```

Описание проведенных тестов

Ввиду отсутствия в задании ссылки на платформу для проверки, тесты были сгенерированы через ЛЛМ (LLM были написаны тесты под задачу, а не под код - так называемое тестирование методом черного ящика).

```
dsa_labs — -zsh — 112x15
ok 01_sample1          0.01 c    7.9 M6
ok 02_sample2          0.01 c    7.9 M6
ok 03_single           0.01 c    7.9 M6
ok 04_no_edges         0.01 c    8.0 M6
ok 05_big_cycle        0.02 c   11.3 M6
ok 10_dense_small     0.01 c    8.1 M6
ok 11_butterfly        0.01 c    7.8 M6
ok big_06_dag_chain    0.02 c   11.0 M6
ok big_07_cycle_chain  0.02 c   10.7 M6
ok big_08_two_big_sccs 0.02 c   11.2 M6
ok big_09_random_sparse 0.03 c   10.7 M6
OK 3.5 Город с односторонним движением lab3/3.5.py 11 тестов, макс 0.03 с (лимит 5 с), 11 M6

Всего запусков: 11, провалов: 0
andrew@MacBook-Pro-Andrew dsa_labs %
```

Выводы

Метки времени post вместе с транспонированием графа позволяют находить компоненты-стоки без явного построения метаграфа. За счет этого наивный алгоритм $O(V^2 + VE)$ сводится к двум обходам за $O(V + E)$.

10 задача.

Оптимальный обмен валюты (2 балла)

Теперь вы хотите вычислить оптимальный способ обмена данной вам валюты c_i на все другие валюты. Для этого вы находите кратчайшие пути из вершины c_i во все остальные вершины.

Дан ориентированный граф с возможными отрицательными весами ребер, у которого n вершин и m ребер, а также задана одна его вершина s . Вычислите длину кратчайших путей из s во все остальные вершины графа.

Формат ввода / входного файла (input.txt). Ориентированный взвешенный граф задан по формату 1.

Ограничения на входные данные.

$1 \leq n \leq 10^3$, $0 \leq m \leq 10^4$, $1 \leq s \leq n$, вес каждого ребра – целое число, не превосходящее по модулю 10^9 .

Формат вывода / выходного файла (output.txt). Для каждой вершины i графа от 1 до n выведите в каждой отдельной строке следующее:

- «*», если пути из s в i нет;
- «-», если существует путь из s в i , но нет кратчайшего пути из s в i (то есть расстояние от s до i равно $-\infty$);
- длину кратчайшего пути в остальных случаях.

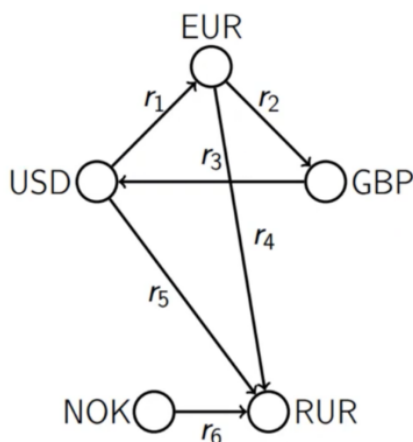
Ограничение по времени. 10 сек.

Ограничение по памяти. 512 мб.

Описание решения

Воспользуемся помощью лекционного материала.

Задача про обмен валюты



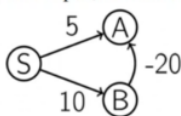
Дано: Граф обмена валют с взвешенными направленными ребрами e_i между некоторыми парами валют с весами r_{e_i} , соответствующими обменному курсу.

Вывод: Максимизируйте

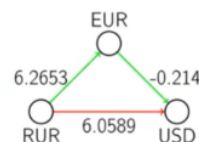
$\prod_{j=1}^k r_{e_j} = r_{e_1} \cdot r_{e_2} \cdot \dots \cdot r_{e_k}$ по путям $p = \langle e_1, e_2, \dots, e_k \rangle$ из долларов США в рубли в графе.

Решено?

- Получается, для решения задачи про обмен валюты нужно всего-лишь:
 - Создать граф обмена валюты с весами r_{e_j} , соответствующими обменным курсам.
 - Заменить r_{e_j} на $(-\log(r_{e_j}))$.
 - Найти кратчайший путь из вершины USD в вершину RUR по алгоритму Дейкстры.
 - Восстановить путь и сделать обмены, соответствующие кратчайшему пути.
- Где алгоритм Дейкстры не работает?
 - Алгоритм Дейкстры основан на том факте, что кратчайший путь из s в t проходит только через вершины, расположенные ближе к s (жадный).
 - Это не относится к графам с отрицательными ребрами:



- Пример:



- $0.013 \times 1.16 = 0.01508 > 0.015$
- $6.2653 - 0.214 = 6.0513 < 6.0589$

Алгоритм Дейкстры не подходит тк имеются отрицательные веса. Необходимо использовать алгоритм Беллмана-Форда - будет по итогу $n - 1$ итерация, на каждой релаксируем все ребра. Этого хватит тк кратчайший путь без циклов содержит не больше $n - 1$ ребра. Если на n -й итерации что-то еще релаксируется то значит есть отрицательный цикл. Но пометить надо не только его вершины, а все достижимые из него, поэтому от найденных вершин запускаем обычный обход по графу. Асимптотика $O(n * m)$

Решение

```
n, m = map(int, input().split())
edges = []
g = [[] for _ in range(n + 1)]

for _ in range(m):
    u, v, w = map(int, input().split())
    edges.append((u, v, w))
    g[u].append(v)

s = int(input())

inf = float('inf')
dist = [inf] * (n + 1)
dist[s] = 0

for _ in range(n - 1):
    changed = False
    for u, v, w in edges:
        if dist[u] != inf and dist[u] + w < dist[v]:
            dist[v] = dist[u] + w
            changed = True
    if not changed:
        break

neg = [False] * (n + 1)
stack = []

for u, v, w in edges:
    if dist[u] != inf and dist[u] + w < dist[v] and not neg[v]:
        neg[v] = True
        stack.append(v)

while stack:
    x = stack.pop()
    for u in g[x]:
        if not neg[u]:
            neg[u] = True
            stack.append(u)

for i in range(1, n + 1):
    if dist[i] == inf:
        print("*")
    elif neg[i]:
        print("-")
    else:
        print(dist[i])
```

Описание проведенных тестов

Ввиду отсутствия в задании ссылки на платформу для проверки, тесты были сгенерированы через ЛЛМ (LLM были написаны тесты под задачу, а не под код - так называемое тестирование методом черного ящика).

```
dsa_labs — -zsh — 112x18
ok 01_sample1          0.01 c    7.8 M6
ok 02_sample2          0.01 c    7.6 M6
ok 03_single_vertex    0.01 c    7.8 M6
ok 04_all_unreachable  0.01 c    7.8 M6
ok 05_source_in_cycle  0.01 c    7.7 M6
ok 06_zero_cycle       0.01 c    7.8 M6
ok 07_huge_negative_chain 0.01 c    8.1 M6
ok 08_huge_positive_chain 0.01 c    8.2 M6
ok 09_reverse_chain    0.09 c    8.2 M6
ok 10_downstream_minus_inf 0.07 c    9.2 M6
ok big_11_max_dag      0.04 c    9.3 M6
ok big_12_max_with_neg_cycle 1.04 c    9.7 M6
ok big_13_random_source 0.03 c    9.2 M6
ok big_14_worst_order  1.11 c    9.6 M6
OK 3.10 Оптимальный обмен валюты lab3/3.10.py 14 тестов, макс 1.11 с (лимит 10 с), 10 M6

Всего запусков: 14, провалов: 0
andrew@MacBook-Pro-Andrew dsa_labs %
```

Выводы

Беллман-Форд работает с отрицательными весами, где алгоритм Дейкстры не работает, и заодно проверяет отрицательные циклы. Отдельно пришлось доработать его обходом, чтобы пометить все вершины с расстоянием минус бесконечность, а не только вершины самого цикла.

15 задача.

Герои (3 балла)

Коварный кардинал Ришелье вновь организовал похищение подвесок королевы Анны; вновь спасти королеву приходится героическим мушкетерам. Атос, Портос, Арамис и д'Артаньян уже перехватили агентов кардинала и вернули украденное; осталось лишь передать подвески королеве Анне. Королева ждет мушкетеров в дворцовом саду. Дворцовый сад имеет форму прямоугольника и разбит на участки, представляющие собой небольшие садики, содержащие коллекции растений из разных климатических зон. К сожалению, на некоторых участках, в том числе на всех участках, расположенных на границах сада, уже притаились в засаде гвардейцы кардинала; на бой с ними времени у мушкетеров нет. Мушкетерам удалось добыть карту сада с отмеченными местами засад; теперь им предстоит выбрать наиболее оптимальные пути к королеве. Для надежности друзья разделили между собой спасенные подвески и проникли в сад поодиночке, поэтому начинают свой путь к королеве с разных участков сада. Двигаются герои по максимально короткой возможной траектории. Марлезонский балет вот-вот начнется; королева не в состоянии ждать героев больше L минут; ровно в начале $L+1$ -ой минуты королева покинет парк, и те мушкетеры, что не успеют к этому времени до нее добраться, не смогут передать ей подвески. На преодоление одного участка у мушкетеров уйдет ровно по минуте. С каждого участка мушкетеры могут перейти на 4 соседние. Требуется выяснить, сколько подвесок будет красоваться на платье королевы, когда она придет на бал.

Формат входных данных (input.txt) и ограничения. Первая строка входного файла INPUT.TXT содержит целые числа N и M ($1 \leq N, M \leq 20$) – размеры сада. Далее идут N строк по M символов в каждом; символы '00' соответствуют участкам, на которых нет засады, символы '1' – участкам, на которых разместились гвардейцы. В $N + 2$ -ой строке теста записано три целых числа: координаты участка, на котором королева будет ждать мушкетёров (Q_x, Q_y) ($1 < Q_x < N, 1 < Q_y < M$) и время в минутах до начала балета ($1 \leq L \leq 1000$). В $N + 3$ -ей строке записаны через пробел целые числа координаты участка, с которого стартует Атос (A_x, A_y) ($1 < A_x < N, 1 < A_y < M$) и количество подвесок, хранящихся у него ($1 \leq P_a \leq 1000$). В $N + 4, N + 5$ и $N + 6$ -ой строках аналогично записаны стартовые координаты и количество подвесок у Портоса, Арамиса и д'Артаньяна.

Формат выходных данных (output.txt). В выходной файл OUTPUT.TXT выведите количество подвесок, которое королева успеет получить у мушкетеров до начала балета.

Ограничение по времени. 1 сек.

Ограничение по памяти. 16 мб.

Проверяем обязательно – на acmp (https://acmp.ru/index.asp?main=task&id_task=846).

Описание решения

Задача сводится к поиску в ширину. Сад - это граф: вершины это свободные участки, ребра между соседними по стороне.

Переход на соседний участок занимает ровно минуту, то есть все ребра весят одинаково. Значит кратчайший путь дает обычный BFS, алгоритм Дейкстры здесь не уместен.

Запускаем BFS не от мушкетеров, а от клетки королевы тк одного обхода хватает на всех. Получается время до каждого участка. Дальше по каждому мушкетеру смотрим его клетку: если она достижима и время не больше L, прибавляем его подвески к ответу.

Асимптотика $O(N * M)$

Решение

```
from collections import deque

n, m = map(int, input().split())
grid = [input() for _ in range(n)]
qx, qy, l = map(int, input().split())

dist = [[-1] * m for _ in range(n)]
dist[qx - 1][qy - 1] = 0
q = deque([(qx - 1, qy - 1)])

while q:
    x, y = q.popleft()
    for dx, dy in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        a, b = x + dx, y + dy
        if 0 <= a < n and 0 <= b < m and grid[a][b] == '0' and
dist[a][b] == -1:
            dist[a][b] = dist[x][y] + 1
            q.append((a, b))

total = 0
for _ in range(4):
    ax, ay, p = map(int, input().split())
    if dist[ax - 1][ay - 1] != -1 and dist[ax - 1][ay - 1] <= l:
        total += p

print(total)
```

Описание проведенных тестов

№1 - №20								Вперед »
ID	Дата	Автор	Задача	Язык	Результат	Тест	Время	Память
25474815	21.09.2026 15:12:06	Жунис Нурдаулет	0022	Python	Running	5	0,062	344 Кб
25474814	21.09.2026 15:12:05	Неизвестный	0021	C++	Compiling			
25474813	21.09.2026 15:12:02	Жунис Нурдаулет	0022	Python	Accepted		0,031	320 Кб
25474812	21.09.2026 15:12:01	Попов	0011	C++	Wrong answer	1		330 Кб
25474811	21.09.2026 15:11:58	Жунис Нурдаулет	0022	Python	Accepted		0,046	316 Кб
25474810	21.09.2026 15:11:53	Северюхин Андрей Аркадьевич	0846	Python	Accepted		0,046	384 Кб
25474809	21.09.2026 15:11:52	Жунис Нурдаулет	0022	Python	Accepted		0,046	344 Кб
25474808	21.09.2026 15:11:41	Ключников Никита Алексеевич	0076	C++	Accepted		0,656	354 Кб
25474807	21.09.2026 15:11:35	Полтавский Александр Алексеевич	0005	Python	Wrong answer	1		316 Кб
25474806	21.09.2026 15:11:33	Бегайдар Нурхан	0108	C++	Accepted		0,015	322 Кб
25474805	21.09.2026 15:11:26	Задорина Дарья Евгеньевна	1120	C++	Accepted		0,03	326 Кб
25474804	21.09.2026 15:11:25	Алим Бахтияр	0001	C++	Wrong answer	1		318 Кб
25474803	21.09.2026 15:11:22	Жунис Нурдаулет	0297	Python	Time limit exceeded	1	2,187	320 Кб
25474802	21.09.2026 15:11:21	Неизвестный	2005	Python	Wrong answer	2	0,031	320 Кб
25474801	21.09.2026 15:11:17	Известный	0637	C++	Accepted		0,03	558 Кб
25474800	21.09.2026 15:11:14	Неизвестный	0021	C++	Wrong answer	2	0,015	318 Кб
25474799	21.09.2026 15:11:07	Борисов Сергей Афанасьевич	0755	C++	Accepted		0,015	322 Кб
25474798	21.09.2026 15:10:59	Аслан	0539	C++	Wrong answer	1		326 Кб
25474797	21.09.2026 15:10:56	Умирбек Нрмейір	1489	C++	Compilation error			
25474796	21.09.2026 15:10:52	Алим Бахтияр	0108	C++	Accepted		0,015	326 Кб
№1 - №20								Вперед »

Выводы

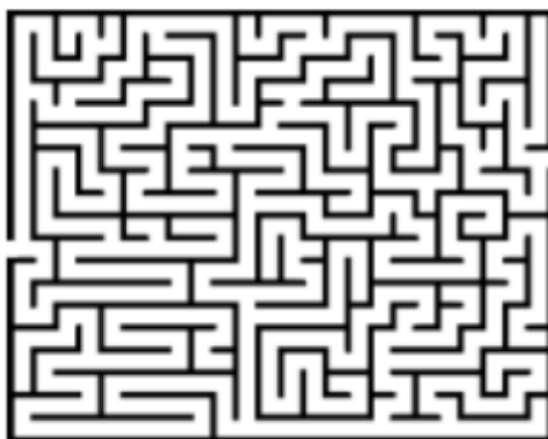
Расстояние в неориентированном графе симметрично, поэтому вместо четырех обходов от мушкетеров хватает одного от королевы. Сложность $O(N * M)$.

Дополнительные задачи

1 задача.

Лабиринт (1 балл)

Лабиринт представляет собой прямоугольную сетку ячеек со стенками между некоторыми соседними ячейками. Вы хотите проверить, существует ли путь от данной ячейки к данному выходу из лабиринта, где выходом также является ячейка, лежащая на границе лабиринта (в примере, показанном на рисунке, есть два выхода: один на левой границе и один на правой границе). Для этого вы представляете лабиринт в виде неориентированного графа: вершины графа являются ячейками лабиринта, две вершины соединены неориентированным ребром, если они смежные и между ними нет стены. Тогда, чтобы проверить, существует ли путь между двумя заданными ячейками лабиринта, достаточно проверить, что существует путь между соответствующими двумя вершинами в графе.



Вам дан неориентированный граф и две различные вершины u и v . Проверьте, есть ли путь между u и v .

Формат ввода / входного файла (input.txt). Неориентированный граф с n вершинами и m ребрами по формату 1. Следующая строка после ввода всего графа содержит две вершины u и v .

Ограничения на входные данные.

$2 \leq n \leq 10^3$, $1 \leq m \leq 10^3$, $1 \leq u, v \leq n$, $u \neq v$.

Формат вывода / выходного файла (output.txt). Выведите 1, если есть путь между вершинами u и v ; выведите 0, если пути нет.

Ограничение по времени. 5 сек.

Ограничение по памяти. 512 мб.

Описание решения

Это Explore, проверка достижимости.

Запускаем обход из вершины u и помечаем все, куда смогли пройти. После обхода смотрим, помечена ли v : помечена - путь есть, не помечена - пути нет.

Граф неориентированный, поэтому каждое ребро кладем в списки смежности обеих вершин. Рекурсия заменена на свой стек.

Асимптотика $O(n + m)$

Решение

```
n, m = map(int, input().split())
g = [[] for _ in range(n + 1)]

for _ in range(m):
    a, b = map(int, input().split())
    g[a].append(b)
    g[b].append(a)

u, v = map(int, input().split())

visited = [False] * (n + 1)
visited[u] = True
stack = [u]

while stack:
    x = stack.pop()
    for y in g[x]:
        if not visited[y]:
            visited[y] = True
            stack.append(y)

print(1 if visited[v] else 0)
```

Описание проведенных тестов

Ввиду отсутствия в задании ссылки на платформу для проверки, тесты были сгенерированы через ЛЛМ (LLM были написаны тесты под задачу, а не под код - так называемое тестирование методом черного ящика).

```
dsa_labs — -zsh — 112x14
ok 01_sample1          0.01 c    7.8 M6
ok 02_sample2          0.01 c    7.7 M6
ok 03_min_connected    0.01 c    7.7 M6
ok 04_min_disconnected 0.01 c    7.7 M6
ok 05_long_path        0.01 c    7.6 M6
ok 06_two_components   0.01 c    7.9 M6
ok 07_star             0.01 c    8.0 M6
ok 08_random_max       0.01 c    7.9 M6
ok 09_isolated_target  0.01 c    7.9 M6
ok 10_cycle            0.01 c    7.9 M6
OK 3.1 Лабиринт       lab3/3.1.py 10 тестов, макс 0.01 с (лимит 5 с), 8 М6

Всего запусков: 10, провалов: 0
andrew@MacBook-Pro-Andrew dsa_labs %
```

Выводы

Вопрос про путь в лабиринте сводится к достижимости и решается одним обходом за $O(n + m)$, сам путь при этом искать не нужно.

2 задача.

Компоненты (1 балл)

Теперь вы решаете сделать так, чтобы в лабиринте не было мертвых зон, то есть чтобы из каждой клетки был доступен хотя бы один выход. Для этого вы находите связанные компоненты соответствующего неориентированного графа и следите за тем, чтобы каждый компонент содержал выходную ячейку.

Дан неориентированный граф с n вершинами и m ребрами. Нужно посчитать количество компонент связности в нем.

Формат ввода / входного файла (input.txt). Неориентированный граф с n вершинами и m ребрами по формату 1.

Ограничения на входные данные.

$1 \leq n \leq 10^3$, $0 \leq m \leq 10^3$.

Формат вывода / выходного файла (output.txt). Выведите количество компонент связности.

Ограничение по времени. 5 сек.

Ограничение по памяти. 512 мб.

Описание решения

Воспользуемся теорией из лекции.

Компоненты связности: алгоритм

Компоненты связности (англ. *Connected Components*)

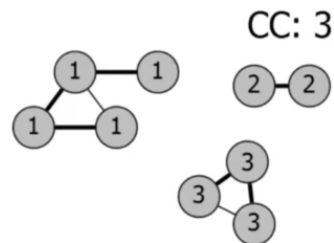
Дано: Граф G

Вывод: Компоненты связности G

Идея:

- `Explore(v)` находит связный компонент v . Нужно повторить, чтобы найти другие компоненты.
- Модифицируем алгоритм обхода в глубину
- Цель модификации – пометить компоненты связности
- В `Explore(v)` добавим нумерацию для каждой вершины `CCnum[v] = cc`
- Каждый новый `Explore` найдет новый компонент связности, в итоге все вершины будут найдены.
- Время выполнения – $O(|V| + |E|)$.

```
1 DFS(G):  
2   for v in V:  
3     visited[v] = False  
4   cc = 1  
5   for v in V:  
6     if not visited(v):  
7       Explore(v)  
8       cc += 1
```



Перебираем вершины по порядку. Если вершина еще не посещена, значит мы наткнулись на новую компоненту - увеличиваем счетчик и запускаем из нее обход, помечая все достижимое.

Обход за один запуск забирает ровно одну компоненту целиком, поэтому количество запусков = ответ.

Асимптотика $O(n + m)$

Решение

```
n, m = map(int, input().split())
g = [[] for _ in range(n + 1)]

for _ in range(m):
    a, b = map(int, input().split())
    g[a].append(b)
    g[b].append(a)

visited = [False] * (n + 1)
count = 0

for s in range(1, n + 1):
    if visited[s]:
        continue
    count += 1
    visited[s] = True
    stack = [s]
    while stack:
        x = stack.pop()
        for y in g[x]:
            if not visited[y]:
                visited[y] = True
                stack.append(y)

print(count)
```

Описание проведенных тестов

Ввиду отсутствия в задании ссылки на платформу для проверки, тесты были сгенерированы через ЛЛМ (LLM были написаны тесты под задачу, а не под код - так называемое тестирование методом черного ящика).

```
dsa_labs — -zsh — 112x13
ok 01_sample          0.01 c    7.6 Mб
ok 02_no_edges        0.01 c    7.9 Mб
ok 03_single_vertex   0.01 c    7.7 Mб
ok 04_path_all        0.01 c    7.9 Mб
ok 05_pairs           0.01 c    7.8 Mб
ok 06_cycle_plus_isolated 0.01 c    7.8 Mб
ok 07_random_max      0.01 c    7.9 Mб
ok 08_clique_plus_isolated 0.01 c    7.8 Mб
ok 09_two_stars       0.01 c    7.9 Mб
OK 3.2 Компоненты     lab3/3.2.py 9 тестов, макс 0.01 с (лимит 5 с), 8 Mб

Всего запусков: 9, провалов: 0
andrew@MacBook-Pro-Andrew dsa_labs %
```

Выводы

Количество компонент связности равно количеству запусков обхода, отдельно ничего считать не надо. Сложность $O(n + m)$.

6 задача.

Количество пересадок (1 балл)

Вы хотите вычислить минимальное количество сегментов полета, чтобы добраться из одного города в другой. Для этого вы строите следующий неориентированный граф: вершины представляют города, между двумя вершинами есть ребро всякий раз, когда между соответствующими двумя городами есть перелет. Тогда достаточно найти кратчайший путь из одного из заданных городов в другой.

Дан неориентированный граф с n вершинами и m ребрами, а также две вершины u и v , нужно посчитать длину кратчайшего пути между u и v (то есть, минимальное количество ребер в пути из u в v).

Формат ввода / входного файла (input.txt). Неориентированный граф задан по формату 1. Следующая строка содержит две вершины u и v .

Ограничения на входные данные.

$2 \leq n \leq 10^5$, $0 \leq m \leq 10^5$, $1 \leq u, v \leq n$, $u \neq v$.

Формат вывода / выходного файла (output.txt). Выведите минимальное количество ребер в пути из u в v . Выведите -1, если пути нет.

Ограничение по времени. 10 сек.

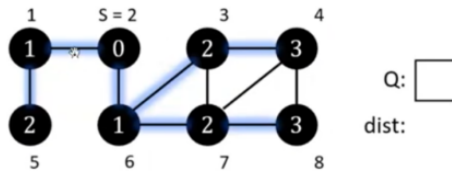
Ограничение по памяти. 512 мб.

Описание решения

Это поиск в ширину из лекции.

Алгоритм поиска в ширину

```
1 BFS(V, adj, S): #Граф G задан количеством вершин V и списком смежности adj
2   for u in range(V): #для каждой вершины графа G
3       dist[u] = inf
4   dist[S] = 0
5   Q = Queue()
6   Q.Enqueue(S)
7   while not Q.is_empty():
8       u = Q.Dequeue()
9       for v in adj[u]: #для каждой вершины в списке смежности u
10          if dist[v] == inf:
11              Q.Enqueue(v)
12              dist[v] = dist[u] + 1
```



Считается количество ребер, то есть все ребра весят одинаково. В таком случае BFS дает кратчайший путь: он обходит вершины по слоям, сначала все на расстоянии 1, потом на расстоянии 2 и так далее.

Ставим $\text{dist}[u] = 0$, кладем u в очередь и каждому новому соседу пишем $\text{dist} + 1$. Если до v так и не дошли, выводим -1 .

Асимптотика $O(n + m)$

Решение

```
from collections import deque

n, m = map(int, input().split())
g = [[] for _ in range(n + 1)]

for _ in range(m):
    a, b = map(int, input().split())
    g[a].append(b)
    g[b].append(a)

u, v = map(int, input().split())

dist = [-1] * (n + 1)
dist[u] = 0
q = deque([u])

while q:
    x = q.popleft()
    for y in g[x]:
        if dist[y] == -1:
            dist[y] = dist[x] + 1
            q.append(y)

print(dist[v])
```

Описание проведенных тестов

Ввиду отсутствия в задании ссылки на платформу для проверки, тесты были сгенерированы через ЛЛМ (LLM были написаны тесты под задачу, а не под код - так называемое тестирование методом черного ящика).

```
dsa_labs — -zsh — 112x14
ok 01_sample1          0.01 c      8.2 Mб
ok 02_sample2          0.01 c      8.2 Mб
ok 03_adjacent          0.01 c      8.2 Mб
ok 04_no_edges          0.01 c      8.2 Mб
ok big_05_long_chain    0.12 c     28.4 Mб
ok big_06_chain_with_shortcut 0.12 c     28.2 Mб
ok big_07_star          0.12 c     23.6 Mб
ok big_08_random_max    0.13 c     25.1 Mб
ok big_09_two_components 0.11 c     26.8 Mб
ok big_10_grid          0.19 c     30.8 Mб
OK 3.6 Количество пересадок lab3/3.6.py 10 тестов, макс 0.19 с (лимит 10 с), 31 Мб
Всего запусков: 10, провалов: 0
andrew@MacBook-Pro-Andrew dsa_labs %
```

Выводы

Пока все ребра одинаковые, кратчайший путь дает обычный BFS за $O(n + m)$, алгоритм Дейкстры при таком раскладе не нужен

7 задача.

Двудольный граф (1.5 балла)

Неориентированный граф называется двудольным, если его вершины можно разбить на две части так, что каждое ребро графа соединяет вершины из разных частей, то есть не существует рёбер между вершинами одной и той же части графа. Двудольные графы естественным образом возникают в задачах, где граф используется для моделирования связей между объектами двух разных типов (например, мальчиками и девочками, или студентами и общежитиями). Альтернативное определение таково: граф двудольный, если его вершины можно раскрасить двумя цветами (например, черным и белым) так, что концы каждого ребра окрашены в разные цвета. Дан неориентированный граф с n вершинами и m ребрами, проверьте, является ли он двудольным.

Формат ввода / входного файла (input.txt). Неориентированный граф задан по формату 1.

Ограничения на входные данные.

$$1 \leq n \leq 10^5, 0 \leq m \leq 10^5.$$

Формат вывода / выходного файла (output.txt). Выведите 1, если граф двудольный; и 0 в противном случае.

Ограничение по времени. 10 сек.

Ограничение по памяти. 512 мб.

Что делать? Адаптируйте поиск в ширину (BFS), чтобы решить эту проблему.

Описание решения

По условию надо адаптировать BFS.

Двудольность это то же самое, что раскраска в два цвета. Красим стартовую вершину в первый цвет, а каждого нового соседа в противоположный цвет.

Если очередной сосед уже покрашен и его цвет совпал с цветом текущей вершины, значит ребро соединяет вершины одной части, и граф не двудольный.

Граф может быть несвязным, поэтому запускаем BFS из каждой непокрашенной вершины.

Асимптотика $O(n + m)$

Решение

```
from collections import deque

n, m = map(int, input().split())
g = [[] for _ in range(n + 1)]

for _ in range(m):
    a, b = map(int, input().split())
    g[a].append(b)
    g[b].append(a)

color = [-1] * (n + 1)
ok = True

for s in range(1, n + 1):
    if color[s] != -1:
        continue
    color[s] = 0
    q = deque([s])
    while q:
        x = q.popleft()
        for y in g[x]:
            if color[y] == -1:
                color[y] = 1 - color[x]
                q.append(y)
            elif color[y] == color[x]:
                ok = False

if ok:
    print(1)
else:
    print(0)
```

Описание проведенных тестов

Ввиду отсутствия в задании ссылки на платформу для проверки, тесты были сгенерированы через ЛЛМ (LLM были написаны тесты под задачу, а не под код - так называемое тестирование методом черного ящика).

```
dsa_labs — -zsh — 112x16
ok 01_sample1          0.01 c    8.3 Mб
ok 02_sample2          0.01 c    8.2 Mб
ok 03_no_edges         0.04 c   15.9 Mб
ok 04_triangle         0.01 c    8.2 Mб
ok 05_even_cycle       0.01 c    8.3 Mб
ok big_06_odd_cycle_last_component 0.13 c   25.3 Mб
ok big_07_even_cycle_1e5 0.13 c   25.2 Mб
ok big_08_odd_cycle_1e5 0.13 c   25.2 Mб
ok big_09_complete_bipartite 0.10 c   13.2 Mб
ok big_10_random_tree  0.14 c   25.5 Mб
ok big_11_tree_plus_triangle 0.13 c   25.5 Mб
ok big_12_grid         0.21 c   28.9 Mб
OK 3.7 Двудольный граф lab3/3.7.py 12 тестов, макс 0.21 с (лимит 10 с), 29 Mб
Всего запусков: 12, провалов: 0
andrew@MacBook-Pro-Andrew dsa_labs %
```

Выводы

BFS красит граф по слоям, поэтому не двудольность видна сразу (ребро внутри одного слоя). Проверка стоит $O(n + m)$.

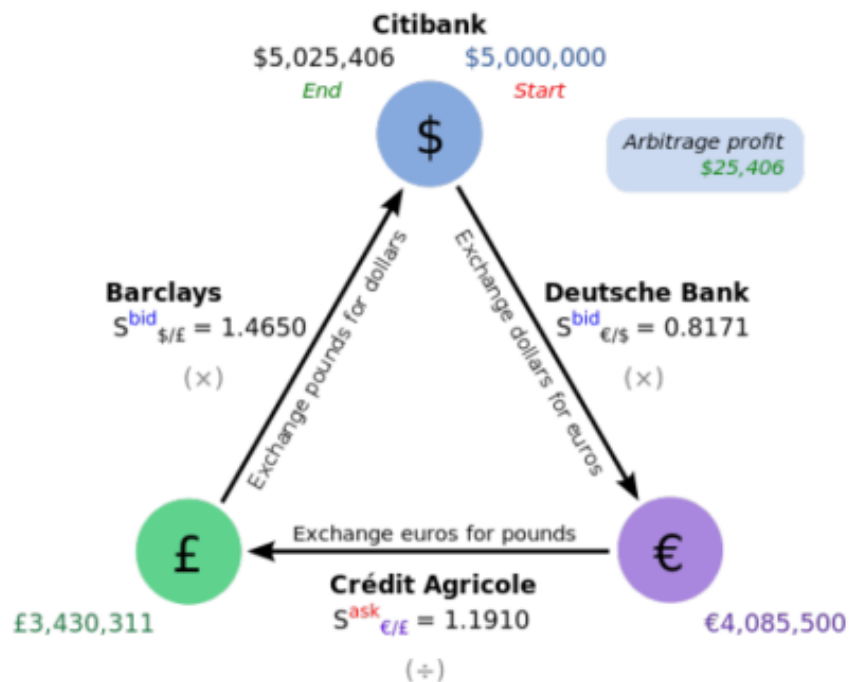
9 задача.

Аномалии курсов валют (2 балла)

Вам дан список валют $c_1, c_2, \dots, c_n$ вместе со списком обменных курсов: r_{ij} – количество единиц валюты c_j , которое можно получить за одну единицу c_i . Вы хотите проверить, можно ли начать делать обмен с одной единицы какой-либо валюты, выполнить последовательность обменов и получить более одной единицы той же валюты, с которой вы начали обмен. Другими словами, вы хотите найти валюты $c_{i_1}, c_{i_2}, \dots, c_{i_k}$ такие, что $r_{i_1, i_2} \cdot r_{i_2, i_3} \cdot \dots \cdot r_{i_{k-1}, i_k} \cdot r_{i_k, i_1} > 1$.

Для этого построим следующий граф: вершинами являются валюты $c_1, c_2, \dots, c_n$, вес ребра из c_i в c_j равен $-\log r_{ij}$. Тогда достаточно проверить, есть ли в этом графе отрицательный цикл. Пусть цикл $c_i \rightarrow c_j \rightarrow c_k \rightarrow c_i$ имеет отрицательный вес. Это означает, что $-(\log c_{ij} + \log c_{jk} + \log c_{ki}) < 0$ и, следовательно, $\log c_{ij} + \log c_{jk} + \log c_{ki} > 0$. Это, в свою очередь, означает, что

$$r_{ij} r_{jk} r_{ki} = 2^{\log c_{ij}} 2^{\log c_{jk}} 2^{\log c_{ki}} = 2^{\log c_{ij} + \log c_{jk} + \log c_{ki}} > 1.$$



Для заданного ориентированного графа с возможными отрицательными весами ребер, у которого n вершин и m ребер, проверьте, содержит ли он цикл с отрицательным суммарным весом.

Формат ввода / входного файла (input.txt). Ориентированный взвешенный граф задан по формату 1.

Ограничения на входные данные.

$1 \leq n \leq 10^3$, $0 \leq m \leq 10^4$, вес каждого ребра – целое число, не превосходящее по модулю 10^4 .

Формат вывода / выходного файла (output.txt). Выведите 1, если граф содержит цикл с отрицательным суммарным весом. Выведите 0 в противном случае.

Ограничение по времени. 10 сек.

Ограничение по памяти. 512 мб.

Описание решения

Данная задача - это обнаружение отрицательного цикла. Воспользуемся теорией:

Алгоритм обнаружения отрицательного цикла

- Выполните $|V|$ итераций алгоритма Беллмана-Форда, сохраните вершину v , на котором выполнялась процедура релаксации на последней итерации.
- Вершина v – достижима из отрицательного цикла.
- Начните с вершины $x \leftarrow v$, следуйте по указателям на предшественника $x \leftarrow \text{prev}[x]$ в цикле $|V|$ раз – обязательно будет в отрицательном цикле
- Следуйте до тех пор пути $x \leftarrow \text{prev}[x]$, до тех пор пока $x == y$ снова.

От задачи 10 отличие в том, что стартовой вершины нет, а цикл может лежать в любой части графа, в том числе недостижимой. Поэтому ставим $\text{dist} = 0$ сразу всем вершинам - это то же самое, что завести лишнюю вершину с нулевыми ребрами во все остальные.

Далее идет n итераций релаксации всех ребер. Если после $n - 1$ итерации что-то еще улучшается, то улучшение можно повторять бесконечно, а это и есть отрицательный цикл. Выводим 1, иначе 0

Асимптотика $O(n * m)$

Решение

```
n, m = map(int, input().split())
edges = [tuple(map(int, input().split())) for _ in range(m)]

dist = [0] * (n + 1)
found = 0

for i in range(n):
    changed = False
    for u, v, w in edges:
        if dist[u] + w < dist[v]:
            dist[v] = dist[u] + w
            changed = True
    if not changed:
        break
    if i == n - 1:
        found = 1

print(found)
```

Описание проведенных тестов

Ввиду отсутствия в задании ссылки на платформу для проверки, тесты были сгенерированы через ЛЛМ (LLM были написаны тесты под задачу, а не под код - так называемое тестирование методом черного ящика).

```
dsa_labs — -zsh — 112x17
ok 01_sample 0.01 c 7.8 M6
ok 02_no_edges 0.01 c 7.7 M6
ok 03_negative_edges_dag 0.01 c 7.7 M6
ok 04_zero_cycle 0.01 c 7.7 M6
ok 05_two_vertex_cycle 0.01 c 7.7 M6
ok 06_cycle_minus_one 0.01 c 7.4 M6
ok 07_unreachable_cycle 0.01 c 7.7 M6
ok 08_long_cycle_n1000 0.12 c 7.9 M6
ok 09_long_cycle_zero 0.12 c 8.0 M6
ok big_10_dag_max 0.04 c 9.3 M6
ok big_11_dag_plus_back_edge 0.04 c 9.3 M6
ok big_12_random_weighted 0.02 c 9.2 M6
ok big_13_worst_order 0.82 c 9.3 M6
OK 3.9 Аномалии курсов валют lab3/3.9.py 13 тестов, макс 0.82 с (лимит 10 с), 9 M6
Всего запусков: 13, провалов: 0
andrew@MacBook-Pro-Andrew dsa_labs %
```

Выводы

Проверка арбитража сводится к поиску отрицательного цикла, а он находится самим фактом лишней релаксации, без явного поиска самого цикла

Вывод по всей работе

В рамках выполнения данной работы познакомились с графами, с DFS и BFS, алгоритмом Дейкстры, алгоритмом Крускала, алгоритмом Беллмана-Форда.